

Dokumentacja modułu regresji liniowej

Dominik Lech

10 maja 2026

1 Założenia projektu

Zadaniem mojej części projektu jest wystawić endpoint HTTP, który pozwoli na trenowanie modelu regresji liniowej na podstawie zadanych plików CSV na różne sposoby – np. z dopuszczeniem różnych automatycznych transformacji, regularyzacja L1 i L2, ze sprawdzeniem wytrenowanego modelu na zbiorze testowym i tak dalej. Mój moduł zapewnia osobie (np. analitykowi) korzystającej z programu opcję zobaczenia, które kolumny są dobrymi predyktorami dla wybranej kolumny, a które są mniej istotne, jak prosty model działa na zbiorze testowym i jaka część wariancji wyjaśnia (wartość R^2).

Projekt zakłada, że dane w CSV są już odpowiednio przygotowane i oczyszczone – tutaj tylko trenuję regresję liniową dla wybranych kolumn z podanego pliku. Dlatego pliki CSV nie mogą mieć brakujących wartości ani zmiennych jakościowych – one powinny być wcześniej zmienione na ilościowe, np. poprzez kodowanie zero-jedynkowe (one-hot encoding).

Dla `method="ols"`, zależnie od celu, zalecana jest standaryzacja danych przed podaniem ich do trenowania – tak żeby miało sens porównywanie wielkości współczynników przy predyktorach. Dla ridge, lasso i elastic net standaryzacja jest **konieczna**, żeby algorytmy mogły zadziałać poprawnie.

2 Wykorzystane biblioteki

Wszystkie potrzebne biblioteki dodano do `requirements.txt`. Najważniejsze:

- `pandas` – do załadowania CSV do Pythona i zarządzania danymi w postaci ramki danych,
- `numpy` – do transformacji,
- `statsmodels` – do trenowania OLS,
- `scikit-learn` – do trenowania ridge i lasso.

3 Struktura projektu

Wszystko znajduje się w katalogu `regresje`, który zawiera:

- `linear_regression.py` – funkcja wywołująca inne funkcje i zwracająca końcowy wynik,
- `validations.py` – odpowiada za walidację danych,
- `apply_transformations.py` – aplikuje transformacje (dodaje nowe kolumny na podstawie istniejących),
- `ols.py` – trenuje model standardowej regresji liniowej,
- `lasso.py` – trenuje model z regularyzacją L1,

- `ridge.py` – trenuje model z regularyzacją L2,
- `elastic_net.py` – trenuje model z regularyzacją L1+L2,
- `base_regression_model.py` – wspólna baza dla modeli regularizowanych,
- `regression_model_factory.py` – fabryka modeli regularizowanych,
- `utils.py` – małe pomocnicze funkcje,
- dokumentacja/ – katalog zawierający dokumentację.

4 Dane wejściowe

4.1 Wymagane pola

- `train_file` – plik, na którym będziemy trenować model.
- `target_column` (string) – nazwa kolumny, którą model będzie przewidywać.
- `mode` (string) – tryb wyboru predyktorów:
 - `"all_parameters"` – wszystkie kolumny oprócz `target_column` są predyktorami,
 - `"include_parameters"` – tylko kolumny podane w `columns` są predyktorami,
 - `"exclude_parameters"` – wszystkie kolumny oprócz `target_column` i podanych w `columns` są predyktorami.

4.2 Pola opcjonalne

- `columns` (string) – lista nazw kolumn oddzielona przecinkami (używana dla `include_parameters` lub `exclude_parameters`).
- `transformations` (string) – lista transformacji w formacie JSON:

```
[{"column": "NAZWA_KOLUMNY", "type": "1/x"},
 {"column": "NAZWA_KOLUMNY", "type": "square"}]
```

Dostępne typy: `"1/x"`, `"square"`, `"log"`.

- `test_file` – opcjonalny plik testowy (musi mieć identyczne kolumny i typy jak `train_file`). Jeśli podany, po trenowaniu model jest testowany na nowych danych.
- `method` (string) – metoda trenowania:
 - `"ols"` – zwykła regresja liniowa,
 - `"lasso"` – regresja z regularyzacją L1 (współczynniki mogą spaść do zera),
 - `"ridge"` – regresja z regularyzacją L2 (współczynniki nie zerują się),
 - `"elastic_net"` – regresja z regularyzacją L1+L2.
- `alpha` (float) – dodatni parametr regularyzacji, wymagany dla `lasso`, `ridge` i `elastic_net`. Im bliższy zeru, tym regularyzacja mniejsza; `alpha=0` jest blokowane – wtedy należy użyć `method="ols"`.
- `l1_ratio` (float) – wymagany tylko dla `method="elastic_net"`, musi należeć do przedziału `[0,1]`. Określa on proporcję regularyzacji L1 i L2: wartości bliższe 1 wzmacniają wpływ L1, a bliższe 0 wzmacniają wpływ L2.

5 Dane wyjściowe

Przykładowa odpowiedź JSON:

```
{
  "parameters": [
    {
      "name": "intercept",
      "coefficient": "0.467012",
      "p_value": "0.000003",
      "standard_error": "0.084878",
      "conf_int_lower": "0.295032",
      "conf_int_upper": "0.638992"
    },
    ..
  ],
  "r_squared": "0.335073",
  "adj_r_squared": "0.155364",
  "f_p_value": "0.082819",
  "prediction_results": {
    "mse": "0.243552",
    "r2": "0.011825"
  },
  "explanations": {
    "warnings": [
      "HIGH_OUTLIER_RATIO",
      "MODEL_NOT_STATISTICALLY_SIGNIFICANT",
      "LIKELY_OVERFITTING"
    ],
    "train_iqr_outliers_count": 16,
    "train_all_rows_count": 48,
    "train_vif": {
      "Pclass": 2.475407,
      ...
    }
  }
}
```

Opisy pól:

- **parameters** – lista estymowanych współczynników.
 - **name** – nazwa parametru (lub **intercept** dla wyrazu wolnego).
 - **coefficient** – wartość współczynnika.
 - **p_value** – poziom istotności (poniżej 0,05 oznacza statystycznie istotny predyktor). Dla ridge/lasso – **null**.
 - **standard_error** – błąd standardowy estymatora. Dla ridge/lasso – **null**.
 - **conf_int_lower** / **conf_int_upper** – dolna i górna granica 95% przedziału ufności. Można to traktować jako zakres wartości, w którym najprawdopodobniej znajduje się prawdziwy wpływ danej kolumny na wynik modelu. Jeśli cały przedział jest powyżej albo poniżej zera, oznacza to, że dana kolumna prawdopodobnie rzeczywiście wpływa na przewidywaną wartość. Dla ridge/lasso – **null**.

- `r_squared` – R^2 na zbiorze treningowym (część wariancji wyjaśnionej przez model).
- `adj_r_squared` – skorygowany R^2 (kara za liczbę predyktorów).
- `f_p_value` – p-wartość testu F dla całego modelu. Dla ridge/lasso – `null`.
- `prediction_results` (obiekt, dostępny tylko jeśli podano `test_file`):
 - `mse` – średniokwadratowy błąd predykcji na zbiorze testowym.
 - `r2` – R^2 na zbiorze testowym. Jeśli jest zauważalnie mniejsze niż R^2 treningowe, sugeruje to overfitting.
- `explanations` (obiekt):
 - `warnings` – lista kodów ostrzeżeń (odpowiedź nadal ma status 200).
 - `train_iqr_outliers_count` – liczba wierszy odstających (IQR) w zbiorze treningowym w przynajmniej jednej kolumnie predyktorów.
 - `train_all_rows_count` – liczba wszystkich wierszy w zbiorze treningowym.
 - `train_vif` – słownik {kolumna: VIF} dla predyktorów na zbiorze treningowym.

Dla metod `lasso`, `ridge` lub `elastic_net` część pól (`p_value`, `standard_error`, `conf_int`, `f_p_value`) będzie miała wartość `null` – jest to oczekiwane.

6 Hipotezy dla p-wartości

- **Test t dla pojedynczego współczynnika** (p_value przy β_j): Wartość p_value pochodzi z testu t, który sprawdza, czy dany predyktor ma istotny liniowy związek z targetem. Hipoteza zerowa H_0 zakłada, że prawdziwy współczynnik β_j jest równy zero - usunięcie tego predyktora nie przekłada się na żadną zmianę wartości przewidywanej. Hipoteza alternatywna H_1 mówi, że $\beta_j \neq 0$, czyli istnieje zależność, której nie można wyjaśnić samym przypadkiem. Jeśli p_value spadnie poniżej przyjętego poziomu istotności (np. 0,05), uznajemy, że mamy wystarczające dowody, by odrzucić H_0 i stwierdzić, że predyktor rzeczywiście wnosi informację do modelu.
- **Test F dla całego modelu** (f_p_value): Globalny test F ocenia, czy jakkolwiek z predyktorów jest użyteczny w przewidywaniu targetu. Hipoteza zerowa H_0 stawia założenie, że wszystkie współczynniki są jednocześnie równe zero ($\beta_1 = \beta_2 = \dots = \beta_p = 0$), co oznacza, że model nie radzi sobie lepiej niż prosta średnia arytmetyczna. Hipoteza alternatywna H_1 zakłada, że wystarczy, że choć jeden ze współczynników jest różny od zera, aby model jako całość miał sens. Niski f_p_value (np. poniżej 0,05) sugeruje więc, że przynajmniej jedna zmienna dobrze przewiduje target.

7 Kody ostrzeżeń

- `CURSE_OF_DIMENSIONALITY` – $n < p$ (mniej rzędów niż predyktorów); trenowanie jest trudne, lepiej użyć regularyzacji a najlepiej to w ogóle zebrać więcej danych.
- `MODERATE_MULTICOLLINEARITY` – co najmniej jeden predyktor ma $5 \leq VIF \leq 10$. Współliniowość predyktorów jest umiarkowana, ale może już być problematyczna.
- `BIG_MULTICOLLINEARITY` – co najmniej jeden predyktor ma $VIF > 10$. Współliniowość predyktorów jest duża, być może warto odrzucić część korelujących ze sobą predyktorów.
- `HIGH_OUTLIER_RATIO` – $\frac{\text{train_iqr_outliers_count}}{\text{train_all_rows_count}} > 0.15$. W zbiorze treningowym jest dużo rzędów odstających - przed treningiem może być warto je odrzucić.

- `LOW_R2` – $R^2 < 0.2$ na zbiorze treningowym. Małe R^2 , sugeruje że relacja między predyktorami a targetem jest słaba.
- `LIKELY_OVERFITTING` – $R^2_{train} - R^2_{test} > 0.1$ (jeśli podano `test_file`). Model ma znacznie lepszy wynik na zbiorze treningowym niż na zbiorze testowym, więc można przypuszczać że jest overfitting i będzie trzeba wprowadzić regularyzację, albo zwiększyć parametr alfa.
- `MODEL_NOT_STATISTICALLY_SIGNIFICANT` – `f_p_value` > 0.05 ; nie ma podstaw do odrzucenia hipotezy H_0 dla testu F całego modelu. Model jest statystycznie nieistotny.

VIF (Variance Inflation Factor) mierzy współliniowość predyktora z pozostałymi; duże wartości sugerują problemy z interpretacją współczynników i stabilnością estymacji, często pomaga regularyzacja.

8 Kody błędów (status 400 Bad Request)

- `INVALID_DATASET` – `train_df` jest pusty lub ma mniej niż 2 kolumny.
- `INVALID_MODE` – podano nieobsługiwany tryb `mode`.
- `TARGET_COLUMN_NOT_FOUND` – brak kolumny `target_column` w pliku treningowym.
- `TEST_DATASET_BAD_STRUCTURE` – `test_file` ma inne kolumny lub typy niż `train_file`.
- `COLUMNS_REQUIRED` – dla trybu `include_parameters` lub `exclude_parameters` brakuje parametru `columns`.
- `UNKNOWN_COLUMNS` – w `columns` podano nazwy kolumn nieistniejące w `train_file`.
- `NO_FEATURE_COLUMNS_SELECTED` – po przetworzeniu nie wybrano żadnej kolumny predyktorów.
- `NON_NUMERIC_COLUMNS_NOT_ALLOWED` – wykryto kolumnę nieliteraryczną.
- `NANS_NOT_ALLOWED` – wykryto brakujące wartości (NaN).
- `INVALID_TRANSFORMATIONS` – niepoprawna struktura JSON dla transformacji.
- `UNKNOWN_TRANSFORMATION_COLUMN` – transformacja dotyczy kolumny nieistniejącej w `train_file`.
- `LOG_TRANSFORMATION_NOT_ALLOWED_FOR_NON_POSITIVE_VALUES` – logarytm z wartości niedodatnich.
- `1/x_TRANSFORMATION_NOT_ALLOWED_FOR_NEGATIVE_VALUES` – dzielenie przez zero lub ujemne (wymagane dodatnie).
- `UNKNOWN_TRANSFORMATION_TYPE` – nieznan typ transformacji.
- `POSITIVE_ALPHA_REQUIRED` – `alpha` nie jest liczbą dodatnią.
- `L1_RATIO_REQUIRED` – dla `method="elastic_net"` brakuje parametru `l1_ratio`.
- `INVALID_L1_RATIO` – `l1_ratio` jest poza zakresem $[0, 1]$.
- `UNKNOWN_METHOD` – podano nieobsługiwaną metodę.
- `COULD_NOT_READ_CSV` – nie udało się wczytać `train_file`.
- `COULD_NOT_READ_TEST_CSV` – nie udało się wczytać `test_file`.

9 Przykładowe wywołanie curl

```
curl --location 'http://127.0.0.1:8000/regressions/linear' \
--form 'mode="all_parameters"' \
--form 'columns="Fare,SibSp,Parch"' \
--form 'target_column="Survived"' \
--form 'train_file=@"train.csv"' \
--form 'test_file=@"test.csv"' \
--form 'transformations="[{"column": "Age", "type": "1/x"}, {"column": "Age", "type": "square"}]"' \
--form 'method="elastic_net"' \
--form 'alpha="0.01"' \
--form 'l1_ratio="0.5"'
```

10 Podsumowanie

Podsumowując, udało mi się utworzyć użyteczne narzędzie pozwalające na wygodne sprawdzanie rezultatów trenowania różnych modeli regresji liniowych w różnych scenariuszach na własnych danych, bez konieczności pisania własnego kodu. Mój moduł będzie wymagał interfejsu graficznego, który będzie łatwy do zaimplementowania dzięki odpowiedniemu opisaniu danych wejściowych, wyjściowych oraz kodów błędów i ostrzeżeń.